

Anonymity Preserving in Lightweight and Decoupled Distributed Authentication for Multihop IoT Networks

Master of Technology
in
Computer Science & Engineering

by
Anup Kulkarni
Roll No.: 244101005

Under the Guidance of
Dr. Manas Khatua



Department of Computer Science & Engineering
Indian Institute of Technology Guwahati
Guwahati - 781039, India

Abstract

The primary objective of this work is to identify a practical and impactful vulnerability within the lightweight and decoupled authentication framework designed for multihop IoT networks, with the goal of extending the scheme to support user anonymity. Through detailed examination of the base protocol, a significant gap was identified: during the authentication phase, the permanent device identity ID_D is transmitted in plaintext over a public channel. This exposure enables any passive adversary to track devices, correlate repeated authentication attempts, and infer network behaviour, thereby creating a major privacy weakness despite the underlying PUF-based and distributed design.

Contents

Abstract	1
Table of contents	2
List of Tables	3
List of Figures	4
1 Introduction	1
2 Literature Survey	2
3 Prerequisites for Objective	4
3.1 Physically Unclonable Function	4
3.2 Hash-based One-Way Accumulator	4
3.3 Core Concept of Anonymity Preserving	5
4 Base Scheme	6
4.1 Overview of the Base Authentication Scheme	6
5 Hardware Implementation	9
5.1 LightSAE Hardware implementation	9
5.2 Hardware Implementation of SAKMS	10
5.3 Hardware Implementation Results Comparison	11
6 Problem Identification	12
6.1 Extending Anonymity Frameworks and Mechanisms to Base scheme	13
6.1.1 Adoption of General Framework 2 (GF2)	13
6.1.2 Mechanism 1: Dynamic Pseudonym Generation	13
6.1.3 Mechanism 2: Dual-State Storage (SF6-UA)	14
6.1.4 Desynchronization Resilience via Dual-State Storage	14
6.1.5 Significance and Recovery Logic	14
6.2 Theoretical Performance Analysis	16
6.2.1 Computational Cost Evaluation	16
6.2.2 Communication Cost Evaluation	16
7 Future Scope	18

List of Tables

1	Comparison of State-of-the-Art IoT Authentication and Anonymity Schemes	3
2	The Benchmark Constants (Base Paper[3])	16
3	COMPUTATIONAL COST: AUTHENTICATION AND KEY EXCHANGE PHASE	16
4	COMMUNICATION COST: AUTHENTICATION AND KEY EXCHANGE PHASE	17

List of Figures

1	Node Enrollment Phase	8
2	Node Authentication Phase	8
3	Node Key Exchange Phase	8
4	LightSAE Hardware Implementation Results.	9
5	SAKMS Implementation Results	10
6	Execution Time Comparison	11
7	Total Energy Comparison	11
8	Node Enrollment Phase	15
9	Node Authentication Phase	15
10	Node Key Exchange Phase	15
11	Authentication and Key Exchange Computational Cost in ms	17
12	Authentication and Key Exchange Communication Cost in bits	17

1 Introduction

The Internet of Things (IoT) has grown very fast in the last few years and has become the main technology behind many smart systems like factories, smart grids, health monitoring and transport networks. These systems use a huge number of small and low-power devices that communicate over wireless links that are open to everyone. Because of this, having a secure and quick authentication method is important to protect IoT setups from attacks like impersonation, replay, MITM, key leaks, or even device capture.

A lot of lightweight authentication methods have been proposed to deal with the low processing power and limited energy of IoT devices. Many works use Physical Unclonable Functions (PUFs), simple hash based challenge-response, and symmetric key ideas because these do not cost much to run. The base protocol used in this work follows the same direction: it uses PUFs, avoids long term key storage, works in a decoupled way across nodes, and supports multi hop verification with very low cost.

But even with all this, there is still a privacy problem. During authentication, the device's real ID is sent openly in the public channel. This leaks the identity and makes it easy for attackers to track the device, link different sessions, or even plan attacks on that device.

Recent research shows that identity privacy in IoT authentication is not just an extra feature but a basic requirement. Many surveys on lightweight anonymous authentication point out that anonymity, untraceability and forward secrecy are important, especially when devices operate in open environments. There are already some anonymity ideas like changing pseudonym IDs every session or using hash chain based temporary identities, but these often bring new issues like desynchronization, large storage, slow identity lookup or high computation. Frameworks like SF3-UA, SF5-UA, SF6-UA and forward secrecy methods like SF7-FS, SF8-FS and SF9-FS show how difficult it is to design a scheme that gives anonymity, unlinkability and still stays safe when messages are lost.

Because of these points, this work focuses on how anonymity features can be added to the chosen PUF based scheme without breaking its lightweight nature. In the current protocol, all authentication messages go through a public channel, so sending the real device ID makes it possible for attackers to follow the device, relate old and new sessions or even pretend to be it. To solve this, the aim is to study anonymous authentication methods from earlier works and find a simple and low cost method that can fit into the base scheme. The goal is to achieve identity anonymity, unlinkability and non traceability while still keeping the protocol light enough for IoT devices.

2 Literature Survey

Recent work in IoT authentication has moved more towards light and distributed methods, mainly because old central systems put too much load on one gateway and also create a single point of failure. Since IoT devices have very low power and limited memory, researchers started looking for ways where many nodes can share the authentication work. Light-SAE [1] is one example of this shift. It lets already authenticated nodes help new ones join using a simple challenge and response idea. It uses TinyJAMBU for light encryption and a modified Diffie–Hellman step to set the session key in multi hop networks. This reduces the load on the gateway. But the problem is that Light-SAE still sends the node identity in plain form and gives no protection for anonymity or unlinkability. So it is not very safe for places where privacy is important.

Another scheme, SAKMS [2], is made for industrial IoT systems that follow the 6TiSCH setup. It uses ECC to do mutual authentication and also uses ECQV certificates. It also improves key management and gives strong forward secrecy. The scheme was checked using ProVerif. But because SAKMS uses many elliptic curve operations, it becomes heavy for very small IoT devices. Also, SAKMS still uses fixed device IDs inside the messages, so the devices can be tracked easily. These issues show that strong security does not always mean privacy and lightweight performance.

The base decoupled PUF-based authentication scheme[3] solves many of these problems by using PUFs to avoid storing long term secrets and by using a hash-based accumulator to check membership. It is decoupled, meaning both parent and non-parent nodes can work as authentication servers, which helps spread the load in multi hop networks. It only uses simple things like hash, XOR and two AES calls, so it is very light to run. But its main weakness is that it sends the device identity (ID.D) directly during authentication. Because all the messages travel over a public channel, an attacker can easily read the identity, follow the device over time, or even plan targeted attacks. So keeping identity private is important.

To solve such privacy issues, many anonymous authentication schemes have been proposed. One secure anonymous IoT authentication protocol[4] uses changing pseudonyms and hash-chain updates so an attacker cannot link two sessions. By replacing fixed IDs with new anonymous IDs every time, the device becomes hard to track. Another scheme for healthcare IoT[5] also focuses on hiding both identity and metadata. It uses nonces and hash masking so that sensitive medical data is safe even on open wireless channels. This shows that anonymity is important even in low power setups.

LAACA[6] is another light anonymous scheme and it uses ECC to protect the identity while doing mutual authentication. But again, ECC operations cost more power and may not fit very small IoT nodes. There is also a survey on lightweight anonymous authentication[7] that lists methods like temporary pseudonyms, PUF-assisted anonymous login,

hidden membership proofs using accumulators, and unlinkable tokens. The survey also explains common problems like desync, pseudonym exhaustion and replay protection, which must be considered when adding anonymity to IoT systems.

Even with all these works, there is still a clear gap. None of the schemes combine all these things together: (i) decoupled authentication, (ii) PUF-based secrecy without long term keys, (iii) very low computation, and (iv) anonymity with unlinkability. Light-SAE[1] and SAKMS[2] do not support privacy. The base scheme[3] has PUFs and is lightweight, but it leaks the device identity. This is why this work focuses on building an anonymous version of the base PUF-based protocol so that no device identity is revealed on the public channel while still keeping the scheme light and distributed.

To show the differences between all these methods clearly, a feature comparison table is added. It lists the use of decoupling, cryptographic tools, anonymity features and the computation and communication cost for all the schemes studied.

Table 1: Comparison of State-of-the-Art IoT Authentication and Anonymity Schemes

Scheme / Ref. No.	Focus Area	Auth Type	Distributed / Multi-hop	Anonymity	Comput. Overhead	Message Overhead
[1] Light-SAE	Large-scale IoT	Challenge–Response + DH	✓	✗	Medium	Medium
[2] SAKMS	Industrial IoT (6TiSCH)	Mutual Authentication (ECC)	✗	✗	High	Low–Medium
[3] Base Paper	Decoupled Distributed Auth PUF-based	PUF Challenge–Response	✓	✗	Very Low	Low
[4] Anonymous IoT Auth (IEEE)	Anonymous Device Authentication	Dynamic Pseudonyms	✗	✓	Medium	Medium
[5] Healthcare Anonymous Auth	IoT Healthcare	Hash-Based Anonymous Auth	✗	✓	Low–Medium	Medium
[6] LAACA	Lightweight Anonymous Key Agreement	ECC-Based Anonymous Auth	✗	✓	Medium–High	Low

3 Prerequisites for Objective

3.1 Physically Unclonable Function

A Physically Unclonable Function (PUF) constitutes a one way physical function that generates a unique response R for a challenge C selected from a pool of challenge space $\mathcal{C}$. PUFs exploit uncontrollable manufacturing variations inherent in integrated circuits, which are practically impossible to replicate. Various types of PUFs have been proposed, including SRAM PUFs, Arbiter PUFs, and Ring Oscillator (RO) PUFs. In this work, a Ring Oscillator PUF is used due to its higher execution speed and lower overhead [?].

A function is considered a valid and secure PUF if it satisfies the following properties:

- **Evaluatable:** A legitimate entity can easily compute a response $R \in \mathcal{R}$ for any given challenge $C \in \mathcal{C}$.
- **Unpredictability:** An adversary cannot predict the response R for any challenge C without physical access to the PUF.
- **Uniqueness:** For two different PUF instances, PUF_1 and PUF_2 , the probability that $\text{PUF}_1(C) = \text{PUF}_2(C)$ for the same challenge C is negligible.
- **Reliability:** For a fixed challenge C , the PUF response $\text{PUF}(C) = R$ remains stable over time and environmental variations.
- **One-wayness:** Given a response R , it is computationally infeasible to determine the corresponding challenge C or invert the PUF.

These properties enable PUFs to serve as intrinsic, hardware-bound secrets for secure key generation, device binding, and authentication in resource-constrained IoT systems.

3.2 Hash-based One-Way Accumulator

A one-way hash accumulator, proposed as an efficient alternative to digital signatures in authentication protocols, provides a compact, non-invertible representation of a set of elements. In such an accumulator, multiple items can be combined using a quasi-commutative operator such that the final accumulated value is independent of the order in which elements are processed.

Formally, a function

$$f : A \times B \rightarrow A$$

is said to be quasi-commutative if, for all $a \in A$ and for all $b, c \in B$,

$$f(f(a, b), c) = f(f(a, c), b). \quad (1)$$

Nyberg introduced a fast accumulated hashing method suitable for lightweight systems. Let N denote the maximum number of accumulable items, and let

$$h : \{0, 1\}^* \rightarrow \{0, 1\}^\ell$$

be a cryptographically secure hash function. Given a set of items:

$$Y = \{y_1, y_2, \dots, y_m\}, \quad m \leq N,$$

each item is first hashed as:

$$Y_i = h(y_i), \quad i = 1, 2, \dots, m.$$

The accumulated hash is then computed as:

$$T_{\text{Acc}} = H(s, Y) = s \bigodot_{i=1}^m h(y_i), \quad (2)$$

where s is an initialization seed and $\odot$ represents the bitwise AND operator. Since the AND operator is commutative and the hash function is one-way, the accumulator inherits quasi-commutative and one-way properties.

Membership Verification: To verify whether an item y_i belongs to the accumulated set, compute:

$$Y_i = h(y_i),$$

and check whether:

$$Y_i \odot T_{\text{Acc}} = T_{\text{Acc}}.$$

The security of the accumulator depends on the difficulty of forging a value Y_i that passes verification. For a security parameter t , the required length r of the accumulated hash is given by:

$$r = N \times e \times t,$$

where e is Euler's (Napier's) constant.

3.3 Core Concept of Anonymity Preserving

In IoT security, anonymity is defined by two key properties: **Untraceability** (concealing the sender's real identity) and **Unlinkability** (preventing the correlation of multiple sessions to a single device). While simply encrypting a static ID protects the credential itself, it does not prevent traffic analysis; if the encrypted value remains constant, an adversary can track the device's activity patterns.

To achieve robust anonymity, modern schemes utilize **Dynamic Pseudonyms**. Instead of a static identifier, the device generates a session-specific alias (PID) using a shared secret or random nonce (e.g., $PID_{i+1} = H(ID \parallel \text{Nonce}_i)$). To an external observer, each authentication request appears as a fresh, random string of bits, completely uncorrelated with previous sessions.

4 Base Scheme

4.1 Overview of the Base Authentication Scheme

The base scheme is designed for multihop IoT networks where newly joined nodes must authenticate themselves securely before communicating with the gateway. Instead of relying on a centralized authority, the gateway distributes the authentication workload by selecting a set of already authenticated IoT nodes to operate as Authentication Servers (AS). Let this set be represented as:

$$AS' = \{AS_1, AS_2, \dots, AS_n\},$$

where the gateway shares a long-term secret key K_{GW-AS} with each AS_i . When a new device D joins the network, the gateway (through the parent node) provides D with the address and identity ID_{AS} of the responsible authenticator. Importantly, the AS may lie at a single-hop or multi-hop distance, and does not need to be the parent node. This decoupling reduces the computational burden on both gateway and parent nodes, improving scalability in dense IoT environments.

The operation of the scheme is divided into two major phases: the **Node Enrollment Phase** and the **Node Authentication and Key Exchange Phase**. In the enrollment phase, node D communicates with its AS over a secure channel. During enrollment, D generates its unique secret y_D , hashes it to obtain:

$$Y_D = H(y_D),$$

and registers this value with the AS. The AS incorporates Y_D into its hash-based accumulator using:

$$T_{Acc} = T_{Acc} \& Y_D.$$

To avoid storing large CRP databases, the AS stores only a lightweight entry consisting of the challenge c_{AS-D} and a masked PUF response:

$$\Phi_D = R_{AS-D} \oplus R_D,$$

while D stores its own challenge c_D . This significantly reduces memory usage and prevents direct leakage of raw PUF responses.

In the **Node Authentication and Key Exchange Phase**, node D authenticates itself to the AS over an insecure public channel. First, D regenerates its PUF response:

$$R_D = PUF_D(c_D),$$

and computes a mask:

$$H(R_D \parallel m_D \parallel ID_D \parallel ts_1),$$

where m_D is the current session-based random obtained during enrollment. The hashed witness:

$$Y_D^H = H(y_D),$$

is XORed with this mask and transmitted to the AS. Because only D and AS know R_D and m_D , the adversary cannot learn Y_D^H from the masked message.

Upon receiving the message, AS reconstructs R_D using:

$$R_D = \Phi_D \oplus R_{AS-D},$$

and regenerates the mask using R_D , m_D , ID_D , and timestamp ts_1 . After unmasking Y_D^H , the AS verifies authenticity by checking:

$$T_{Acc} \& Y_D^H = T_{Acc}.$$

If equality holds, the AS confirms that D was previously registered during enrollment.

The AS then generates a new session-based random:

$$m_{new} = H(n_1),$$

where n_1 is freshly chosen. This replaces m_D for the next session, providing forward security. The session key for secure communication between the gateway and the node is computed as:

$$K_{GW-D} = H(R_D \parallel m_{new}).$$

To securely communicate m_{new} to D , AS masks it using:

$$H(Y_D^H \parallel m_D \parallel R_D \parallel ID_{AS} \parallel ID_D \parallel ts_2).$$

After unmasking and updating its state, D also computes K_{GW-D} . Meanwhile, AS constructs an authentication token $auth_token_D$ containing ID_D , ID_{AS} , timestamp ts_{auth} , and the session key K_{GW-D} , encrypted under the shared key K_{GW-AS} , and sends it to the gateway.

This completes the authentication and key exchange process. The scheme achieves decoupled and distributed authentication, PUF-based security, lightweight computation, low storage overhead, dynamic key updates, and protection against replay, impersonation, and key-exposure attacks.

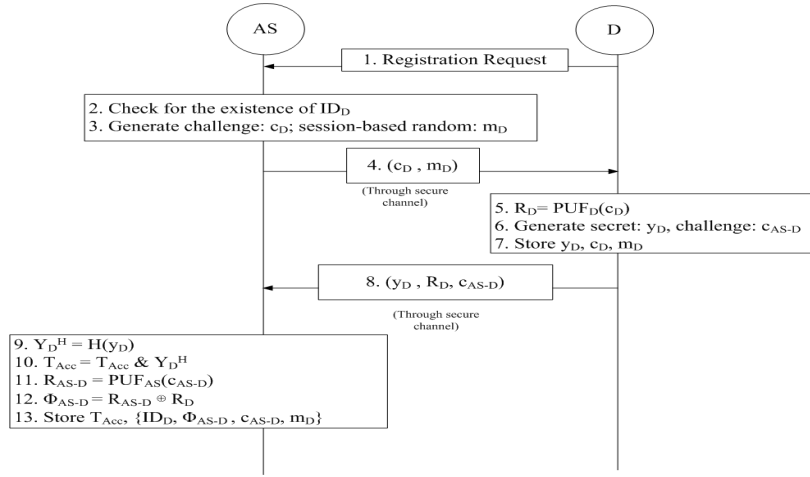


Figure 1: Node Enrollment Phase

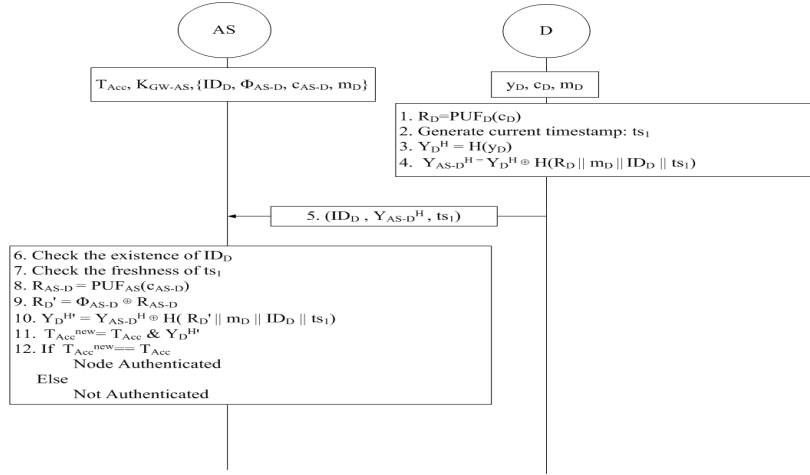


Figure 2: Node Authentication Phase

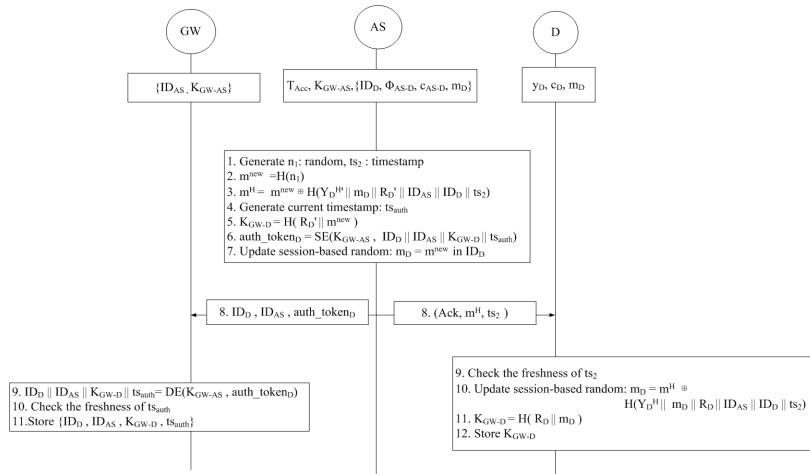


Figure 3: Node Key Exchange Phase

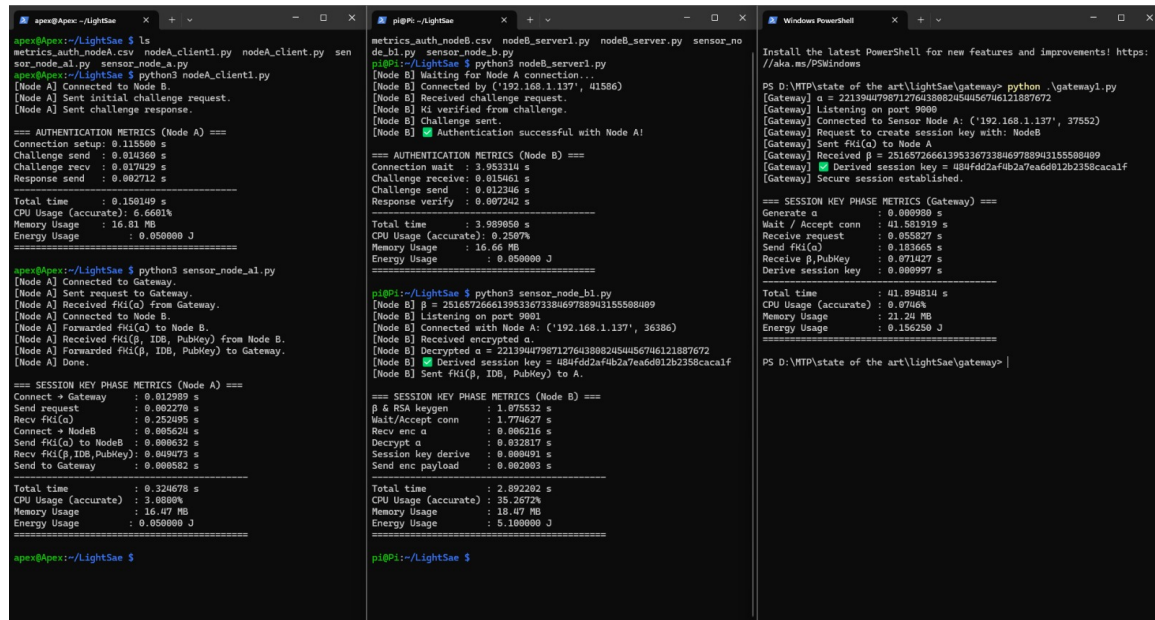
5 Hardware Implementation

5.1 LightSAE Hardware implementation

The Light-SAE protocol was tested on a small setup that used two Raspberry Pi 3B+ boards and one laptop working as the gateway. One Raspberry Pi acted like an authenticator node (a node that was already trusted in the network), and the second Raspberry Pi acted as a new device trying to join. The laptop ran the Light-SAE gateway part, which gives out the initial parameters, checks the responses from the authenticator, and also helps in setting up the session key.

Both Raspberry Pis had Python installed along with lightweight cryptographic libraries like TinyJAMBU and Elephant80, and we wrote small socket-based programs to send messages between them. When the system ran, Node A created the challenges and checked the answers coming from Node B, while the gateway handled the gateway messages described in the original Light-SAE design. The modified Diffie–Hellman exchange and the signature operations were also carried out between the nodes so that they could generate the session key for secure data transfer.

Overall, the protocol worked well on the Raspberry Pi 3B+. The DH key exchange and digital signature checks did add some delay and CPU load, which we could notice during execution. The message logs and packet captures confirmed that the three-way interaction between the Gateway → Authenticator → Joining Node was happening correctly. This allowed us to later compare Light-SAE's behaviour with SAKMS and also with the base scheme.



```
apex@Apex:~/LightSae $ ls
metrics_auth_nodeA.csv  nodeA_client1.py  nodeA_client.py  sensor_nodeA.py
apex@Apex:~/LightSae $ python3 nodeA_client1.py
[Node A] Connected to Node B.
[Node A] Sent initial challenge request.
[Node A] Sent challenge response.

=== AUTHENTICATION METRICS (Node A) ===
Connection setup: 0.115508 s
Challenge send : 0.013368 s
Challenge recv : 0.017429 s
Response send : 0.002712 s

Total time : 0.158149 s
CPU Usage (accurate): 6.6601%
Memory Usage : 16.81 MB
Energy Usage : 0.050000 J

apex@Apex:~/LightSae $ python3 sensor_nodeA1.py
[Node A] Connected to Gateway.
[Node A] Sent request to Gateway.
[Node A] Received fki(a) from Gateway.
[Node A] Connected to Node B.
[Node A] Forwarded fki(a) to Node B.
[Node A] Received fki(b, IDB, PubKey) from Node B.
[Node A] Forwarded fki(b, IDB, PubKey) to Gateway.
[Node A] Done.

=== SESSION KEY PHASE METRICS (Node A) ===
Connect * Gateway : 0.012989 s
Send request : 0.002279 s
Recv fki(a) : 0.252495 s
Connect * NodeB : 0.005620 s
Send fki(a) to NodeB : 0.000632 s
Recv fki(b, IDB, PubKey) : 0.049473 s
Send to Gateway : 0.000582 s

Total time : 0.324678 s
CPU Usage (accurate): 3.0880%
Memory Usage : 16.47 MB
Energy Usage : 0.050000 J

apex@Apex:~/LightSae $

pi@Pi:~/LightSae $ metrics_auth_nodeB.csv nodeB_server1.py nodeB_server.py sensor_nodeB.py
pi@Pi:~/LightSae $ python3 nodeB_server1.py
[Node B] Waiting for Node A connection...
[Node B] Connected by ('192.168.1.137', 41586)
[Node B] Received challenge request.
[Node B] ki verified from challenge.
[Node B] Challenge sent.
[Node B] Authentication successful with Node A!

=== AUTHENTICATION METRICS (Node B) ===
Connection wait : 3.953314 s
Challenge receive: 0.015461 s
Challenge send : 0.012346 s
Response verify : 0.007242 s

Total time : 3.989059 s
CPU Usage (accurate): 0.2507%
Memory Usage : 16.66 MB
Energy Usage : 0.050000 J

pi@Pi:~/LightSae $ python3 sensor_nodeB1.py
[Node B] Listening on port 9080
[Node B] Connected with Node A ('192.168.1.137', 36386)
[Node B] Received encrypted a.
[Node B] Decrypted a = 221394479871276438882454456746121887672
[Node B] Derived session key = 4B84fdd2af4b2a7ea6d012b2335cacalf
[Node B] Sent fki(b, IDB, PubKey) to A.

=== SESSION KEY PHASE METRICS (Node B) ===
β & RSA keygen : 1.075532 s
Wait/Accept conn : 1.774627 s
Recv enc a : 0.006216 s
Decrypt a : 0.032817 s
Session key derive : 0.000491 s
Send enc payload : 0.002083 s

Total time : 2.892282 s
CPU Usage (accurate): 35.2672%
Memory Usage : 18.47 MB
Energy Usage : 5.100000 J

pi@Pi:~/LightSae $

PS D:\MTP\state of the art\LightSae\gateway> python .\gateway1.py
[Gateway] a = 221394479871276438882454456746121887672
[Gateway] Listening on port 9080
[Gateway] Connected to Sensor Node A: ('192.168.1.137', 37552)
[Gateway] Request to create session key with: NodeB
[Gateway] Sent fki(a) to Node A
[Gateway] Received β = 251657266613953367338469788903155508489
[Gateway] Derived session key = 4B84fdd2af4b2a7ea6d012b2335cacalf
[Gateway] Secure session established.

=== SESSION KEY PHASE METRICS (Gateway) ===
Generate a : 0.000980 s
Wait / Accept conn : 41.581919 s
Receive request : 0.055327 s
Send fki(a) : 0.183665 s
Receive β, PubKey : 0.071427 s
Derive session key : 0.000997 s

Total time : 41.894814 s
CPU Usage (accurate): 0.0746%
Memory Usage : 21.24 MB
Energy Usage : 0.156250 J

PS D:\MTP\state of the art\LightSae\gateway>
```

Figure 4: LightSAE Hardware Implementation Results.

5.2 Hardware Implementation of SAKMS

The SAKMS protocol was tested using a three-device setup that had two Raspberry Pi 3B+ boards and a laptop acting as the Certificate Authority (CA). In this setup, the laptop generated the certificates and the key pairs needed by the two IoT nodes. Each Raspberry Pi functioned as a separate 6TiSCH device and followed the mutual authentication steps given in the SAKMS protocol. To support the elliptic-curve computations used in the original design, both Raspberry Pis were installed with Python cryptographic libraries that handle ECC operations.

During system initialization, the CA created the identity keys and implicit certificates for both devices. These were securely moved to each Raspberry Pi, matching the 6TiSCH trust model where a central CA provides all device credentials before deployment. After provisioning, the devices exchanged their certificates and public identity information directly. Each Raspberry Pi reconstructed the peer's public key using the ECQV method, validated the received certificate, and carried out the ECC-based challenge–response steps required for mutual authentication. These computations were implemented using Python's `ecdsa` library and OpenSSL bindings. Everything ran correctly on the Raspberry Pi 3B+, although the ECC operations caused noticeable computation delay because elliptic-curve arithmetic is heavy for low-power devices. After both devices verified each other's credentials, they independently derived the shared session key by running the elliptic-curve Diffie–Hellman-like computation defined in SAKMS. Overall, the hardware experiment showed that while SAKMS provides strong security guarantees and formal certificate-based authentication, its heavy use of ECC results in clear computational overhead on constrained IoT hardware such as Raspberry Pi 3B+.

=== Node A (REGISTRATION) METRICS ===	
Generate ID/Proof	: 0.396805 s
Connect to CA	: 0.007895 s
Send request	: 0.000214 s
Receive response	: 0.023620 s
Verify/Derive d_A	: 0.409474 s

Total operations time	: 0.837408 s
Estimated Energy	: 2.9399 J

Peak CPU Usage (Accurate)	: 99.20 %
Peak CPU Usage (Normalized)	: 24.80 %
Peak Memory (process)	: 16.17 MB
=====	

=== CA (Gateway) METRICS ===	
Wait / Accept conn	: 4.327694 s
Receive request	: 0.003261 s
Generate Certificate	: 0.010145 s
Send response	: 0.000098 s

Total operations time	: 4.340997 s
Estimated Energy	: 121.7775 J

Peak CPU Usage (Accurate)	: 82.70 %
Peak CPU Usage (Normalized)	: 3.76 %
Peak Memory (process)	: 20.04 MB
=====	
[CA] 2025-11-19 12:00:58 Server closed	

(a) SAKMS Registration Phase

=== Node A (AUTHENTICATION) METRICS ===	
Load Credentials	: 0.390200 s
Connect to Node B	: 0.105761 s
Generate MIC_A	: 0.377195 s
Send request	: 0.000355 s
Receive response	: 0.010427 s
Derive Key / Verify MIC_B	: 0.413647 s

Total operations time	: 2.113492 s
Estimated Energy	: 7.3972 J

Peak CPU Usage (Accurate)	: 238.00 %
Peak CPU Usage (Normalized)	: 59.50 %
Peak Memory (process)	: 16.33 MB
=====	

=== Node B (AUTHENTICATION) METRICS ===	
Wait / Accept conn	: 7.074372 s
Receive request	: 0.307345 s
Verify MIC_A	: 0.010026 s
Derive Key / Gen MIC_B	: 0.779729 s
Send response	: 0.000265 s

Total operations time	: 8.251730 s
Estimated Energy	: 28.8811 J

Peak CPU Usage (Accurate)	: 147.00 %
Peak CPU Usage (Normalized)	: 36.00 %
Peak Memory (process)	: 16.00 MB
=====	

(b) SAKMS Authentication and Key Exchange Phase

Figure 5: SAKMS Implementation Results

5.3 Hardware Implementation Results Comparison

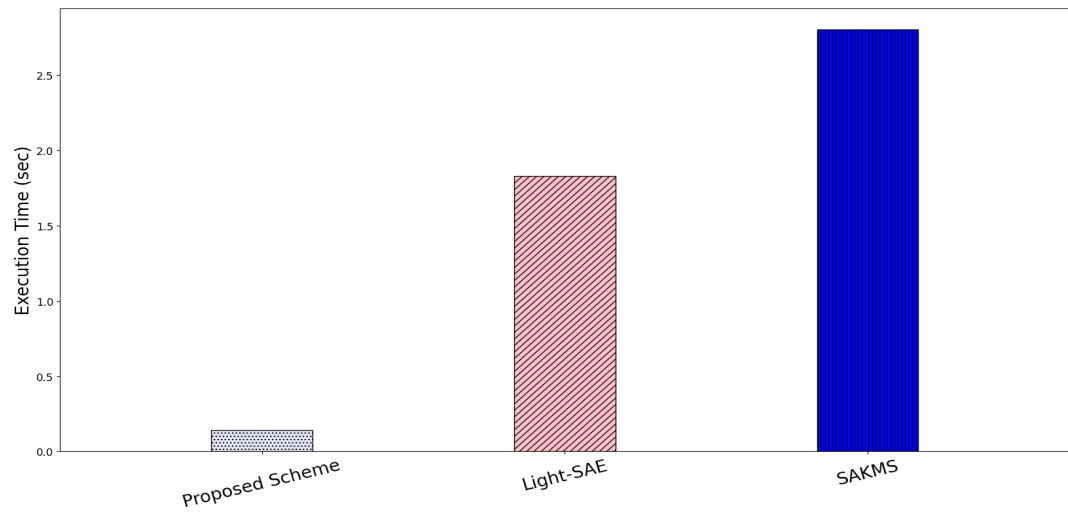


Figure 6: Execution Time Comparison

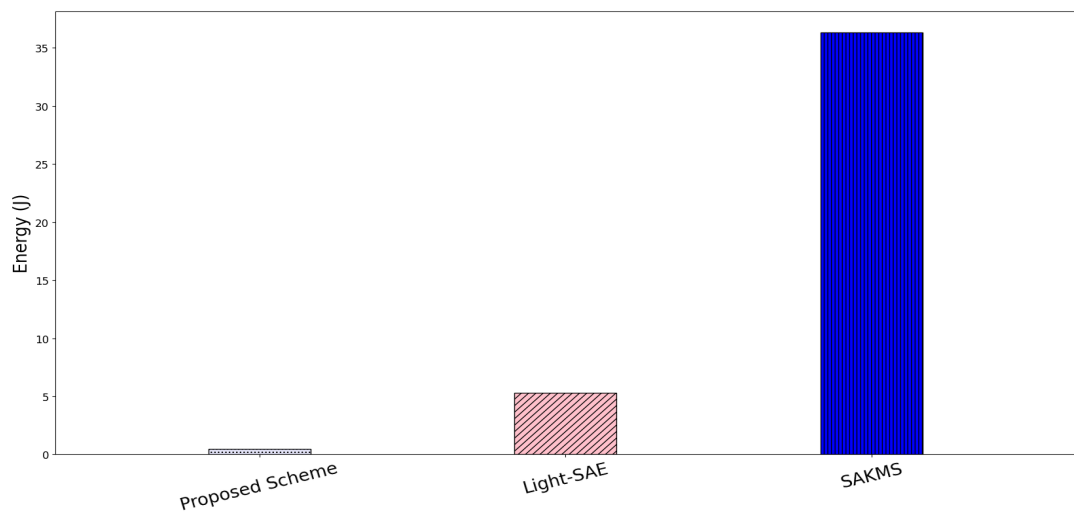


Figure 7: Total Energy Comparison

6 Problem Identification

In the current decoupled and distributed PUF-based authentication scheme, a new IoT device D proves itself to an Authentication Server (AS) using only light operations like hashing, XOR, some small bitwise steps, and the PUF responses that the device generates. This setup avoids storing long-term keys, keeps the computation very low, and spreads the authentication work across many nodes. But even with all these advantages, there is still a major privacy problem that has not been fixed yet. During the authentication process, the real device identity ID_D is sent openly on the wireless network, and this wireless path is not secure.

Since IoT networks usually operate in open and sometimes hostile environments, a passive attacker who is just listening to the traffic can easily capture these identity messages. Even if other fields in the packet are protected, sending ID_D in plain text is enough for an attacker to link different authentication attempts from the same device. Over time, this can allow long-term tracking, behaviour profiling, learning the network structure, or even targeting a particular node for jamming. In a multihop network, the problem becomes even worse because these packets move across many intermediate nodes, giving an attacker several points where they can observe the device identity.

So, although the base scheme is correct and is able to stop common attacks like replay, impersonation and key-exposure, it does not give anonymity and it does not stop linkability. The fact that ID_D is directly leaked becomes a serious weakness that affects device privacy, overall network secrecy, and even the reliability of the system. This gap creates the need for an improved authentication method that hides real device identities, avoids linking across sessions, and adds anonymity without breaking the lightweight and distributed design of the original scheme.

6.1 Extending Anonymity Frameworks and Mechanisms to Base scheme

To resolve the user traceability vulnerability inherent in the Base Scheme, the revised protocol integrates specific frameworks and cryptographic mechanisms derived from the literature[7]. This section details the adoption of the General Framework 2 (GF2) and the specific implementation of Dynamic Pseudonyms and Dual-State Storage SF6 UA[7].

6.1.1 Adoption of General Framework 2 (GF2)

Based on the comparative analysis of authentication taxonomies, the General Framework 2 (GF2) was adopted as the extension for the revised scheme. GF2 is characterized by its ability to achieve User Anonymity, Forward Secrecy, and Desynchronization Resistance simultaneously within a minimal two-round communication structure.

Unlike the Base Scheme, which prioritized only mutual authentication and key exchange, the implementation of GF2 introduces a stateful identity management layer at the Authentication Server (AS). This allows the scheme to transition from a Static Identity Model (susceptible to tracking) to a Dynamic Identity Model (untraceable) without necessitating the heavy computational load of Public Key Infrastructure (PKI).

6.1.2 Mechanism 1: Dynamic Pseudonym Generation

To implement the anonymity requirement of GF2, the revised scheme incorporates a Dynamic Pseudonym Mechanism. This mechanism draws inspiration from the LAACA Scheme, which demonstrates that secure anonymity in fog/IoT environments can be achieved using lightweight Hash and XOR operations rather than heavy encryption.

In the Base Scheme, the device identity (ID_D) was transmitted in plaintext. In the revised implementation, ID_D is replaced by a dynamic pseudonym (PID) derived from the session-specific random parameter (m_D).

The pseudonym generation logic is defined as:

$$PID_i = H(ID_D \parallel m_D)$$

Where:

- $H(\cdot)$ is the one-way collision-resistant hash function used in the Base Scheme.
- ID_D is the static real identity.
- m_D is the session-based random nonce shared between the Device (D) and the AS.

By tying the identity to m_D , the pseudonym automatically rotates after every successful session update. To an external adversary, the transmitted identifier appears as a random 256-bit hash that changes unpredictably with every session, thereby ensuring **Unlinkability** and **Untraceability**.

6.1.3 Mechanism 2: Dual-State Storage (SF6-UA)

The transition to dynamic pseudonyms introduces the risk of desynchronization, where a lost update message causes the device and AS to hold different values for the current identity. To avoid this, the revised scheme implements the Sub-Framework 6 (SF6-UA) mechanism from the referenced survey[7]. To enhance the robustness of the authentication process, storage structure at the Authentication Server (AS) is redesigned to follow a *forgiving memory* model. Instead of relying on a single pseudonym for lookup, the AS maintains two pseudonym states for each registered device.

- **Current State** (PID_{curr}): This entry stores the pseudonym that the device is expected to present in the next authentication session. It is generated using the most recent session update derived from the new randomness m^{new} .
- **Old State** (PID_{old}): This entry retains the pseudonym from the previous successful session, which is computed using the earlier session random value m_D .

6.1.4 Desynchronization Resilience via Dual-State Storage

A desynchronization event typically occurs when the final acknowledgment message of a session—containing the parameters for the next session (e.g., m^{new})—is lost due to network instability or intercepted by an active adversary. In a standard “single-state” architecture, the Server updates its record to the new identity (PID_{new}) immediately after generating the message. If the Device fails to receive this update, it retains the old identity (PID_{old}). Consequently, in the subsequent session, the Device attempts to authenticate using PID_{old} , which the Server no longer recognizes, leading to a permanent **Denial of Service (DoS)** until re-registration is performed.

6.1.5 Significance and Recovery Logic

The significance of this mechanism lies in its ability to provide **automatic self-recovery** from packet loss without compromising security or requiring complex re-handshaking protocols. The AS logic handles incoming authentication requests as follows:

Synchronization: If the AS receives a pseudonym matching $PID_{current}$, it confirms that the previous update was successful and proceeds with the new session keys.

Desynchronization Recovery: If the AS receives a pseudonym matching PID_{old} , it infers that the Device did not receive the last update. Instead of rejecting the user, the AS authenticates using the old credentials and **re-transmits** the update message.

This approach ensures that the scheme remains robust against *Desynchronization Attacks*, where an adversary intentionally blocks update messages to disrupt the network. By retaining the immediate past state, the system guarantees high availability and resilience.

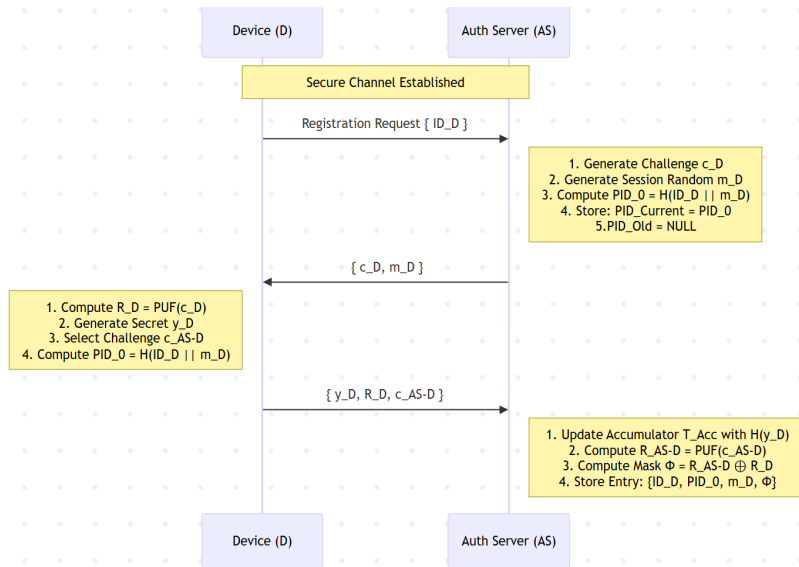


Figure 8: Node Enrollment Phase

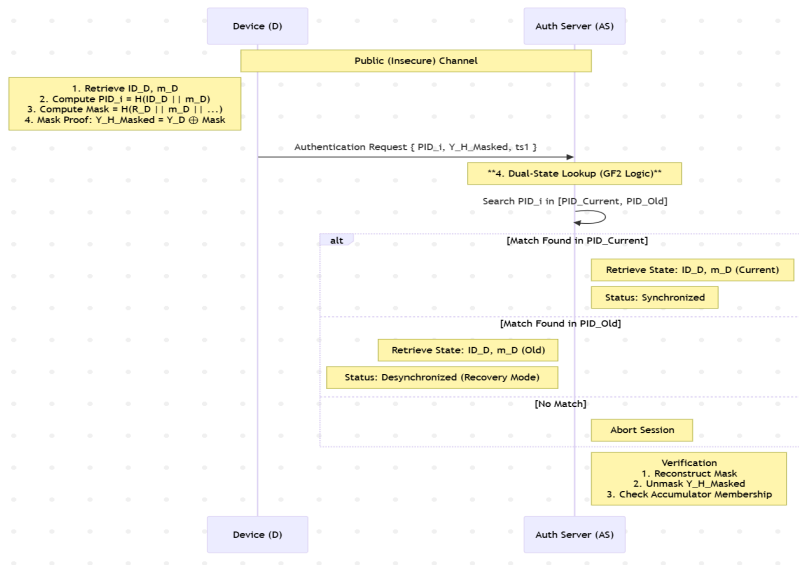
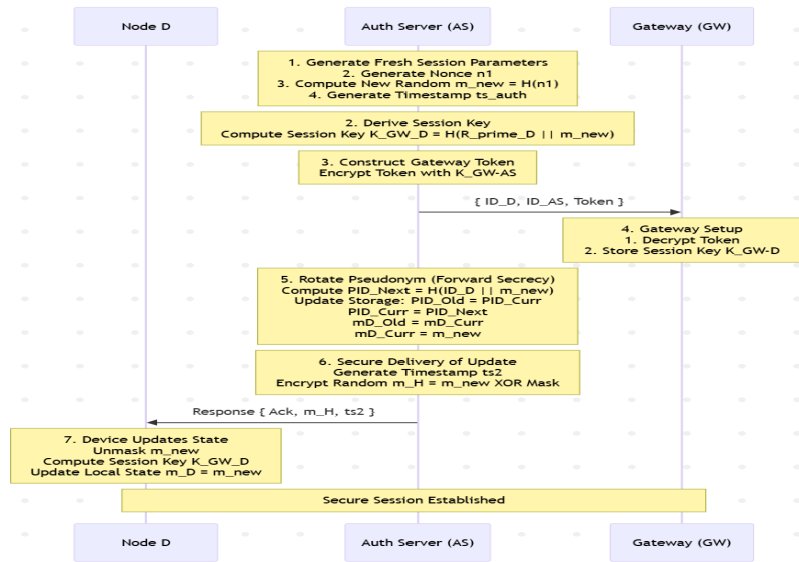


Figure 9: Node Authentication Phase



15
Figure 10: Node Key Exchange Phase

6.2 Theoretical Performance Analysis

6.2.1 Computational Cost Evaluation

The table below provides a comparative summary of the computational cost associated with the proposed Revised Scheme and other SOTA authentication protocols. All execution times are calculated using standard benchmark values for constrained IoT hardware (OpenMote). The results indicate that the Revised Scheme introduces only a negligible overhead compared to the Base Scheme, despite adding anonymity support. This efficiency is achieved through the use of lightweight operations such as hashing and XOR.

Table 2: The Benchmark Constants (Base Paper[3])

Operation	Symbol	Time (ms)	Description
PUF Response	T_{puf}	0.0522	Physical Unclonable Function response generation.
Hash	T_{hash}	0.0353	SHA-256 hashing operation used in authentication and masking.
AES	T_{aes}	0.0867	128-bit symmetric encryption or decryption.
ECC	T_{ecc}	0.9973	Elliptic curve point multiplication or addition.
Random Generation	T_{rand}	0.0018	Random number generation (RNG) for session values.

Table 3: COMPUTATIONAL COST: AUTHENTICATION AND KEY EXCHANGE PHASE

Scheme	Computational Cost Formula	Total Time (ms)
Base Scheme	$2T_{\text{puf}} + 8T_{\text{hash}} + 2T_{\text{aes}} + T_{\text{rand}}$	0.56 (Baseline)
Revised Scheme	$2T_{\text{puf}} + 10T_{\text{hash}} + 2T_{\text{aes}} + T_{\text{rand}}$	0.63 (+12.56%)
LightSAE	$14T_{\text{aes}}$	1.21 (+115.9%)
SAKMS	$6T_{\text{hash}} + 4T_{\text{ecc}}$	4.20 (+647.5%)

6.2.2 Communication Cost Evaluation

This comparison uses Base Scheme as the baseline. The values for LightSAE and SAKMS are taken directly from the comparison table in provided base paper, while the Revised Scheme value is calculated based on the additional data required for anonymity (**replacing the 32-bit ID with a 256-bit Hash**).

Table 4: COMMUNICATION COST: AUTHENTICATION AND KEY EXCHANGE PHASE

Scheme	Total Cost (bits)	Overhead Base (bits)	vs. Performance Impact
Base Scheme	928	0	Base-line
RevisedScheme	1152	+224	(~24%)
LightSAE	1536	+608	(~66%)
SAKMS	2612	+1684	(~181%)

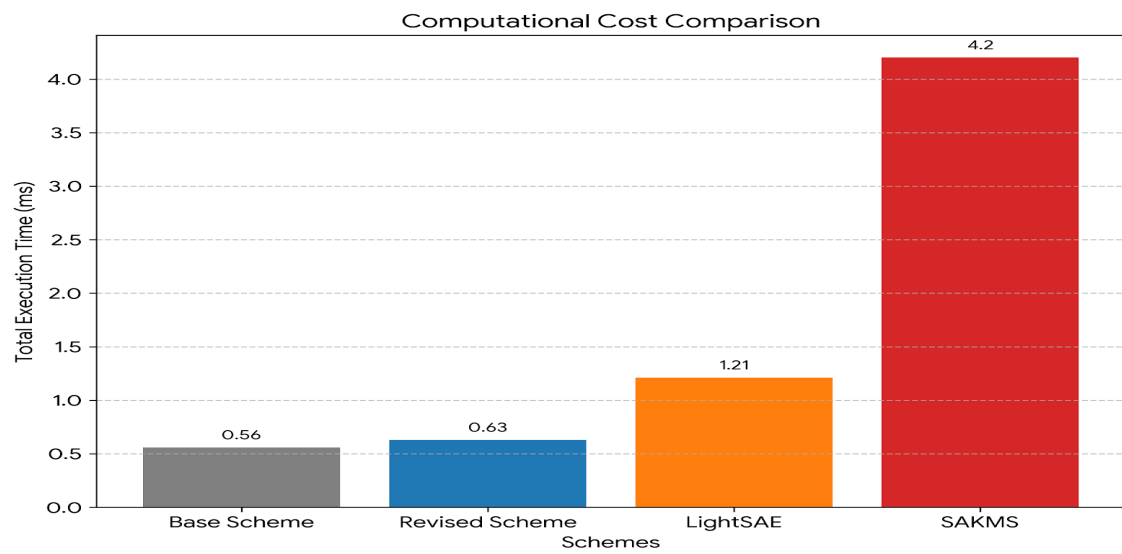


Figure 11: Authentication and Key Exchange Computational Cost in **ms**

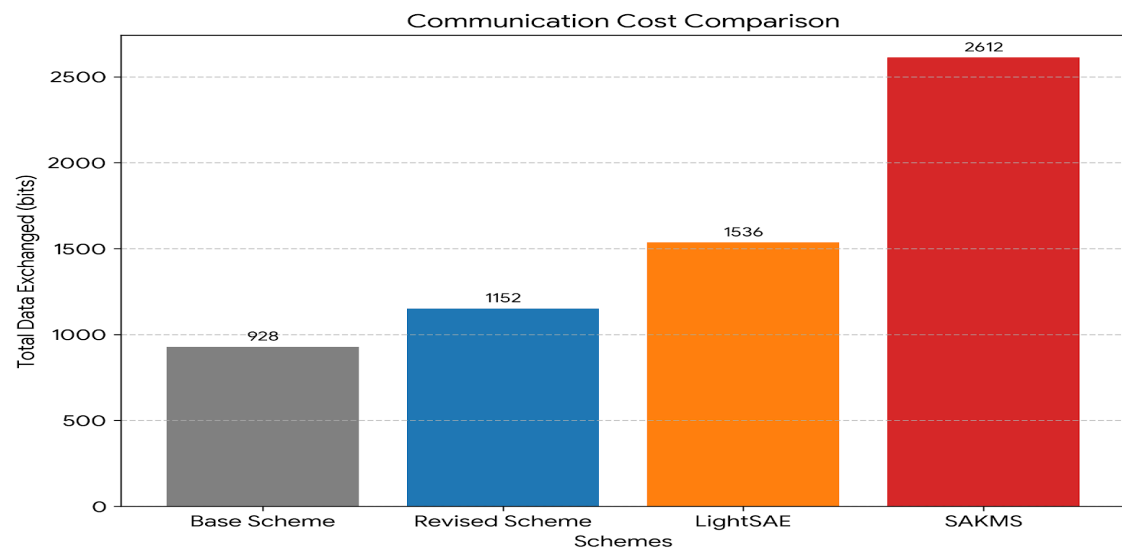


Figure 12: Authentication and Key Exchange Communication Cost in **bits**

7 Future Scope

The anonymity idea discussed in this report is mainly a conceptual extension to the base authentication protocol. It tries to fix the anonymity problem and also the desynchronization issue at a theory level. But even with these improvements, there are still a few steps left before the scheme can be called fully complete or ready to deploy.

The informal checks suggest that the revised scheme should handle normal attack models, but the full formal verification has not been finished yet.

- **ProVerif Verification:** A basic ProVerif model has been written that includes the dynamic pseudonym update logic and the dual-state lookup method. However, the model has not been executed or fully verified yet. This step will help find if an attacker can still break the system or steal the identity or session key in ways we did not expect.
- **Performance Testing on Real Hardware:** All the performance values shown in this report are theoretical. The scheme still needs to be coded properly and tested on real low-power devices like OpenMote-STM or Arduino boards. This will help measure the exact time needed for the extra hash operations and the dual-state lookup and confirm that these steps do not slow down the protocol too much in real networks.
- **Storage Issues on Authentication Servers:** The dual-state storage helps against desynchronization, but it also doubles the memory needed for each device at the Authentication Server. Since the current and old pseudonym states (*PID_curr* and *PID_old*) must both be stored, the memory per user increases. This can be a problem for low-power AS nodes with limited space. Future work should explore ways to reduce the memory use, such as deleting the old state after a timeout or using a smarter storage rule.

References

- [1] P. Rosa, A. Souto, and J. Cecilio, “Light-SAE: A Lightweight Authentication Protocol for Large-Scale IoT Environments,” *IEEE Transactions on Network and Service Management*, 2023.
- [2] W. Yang, C. Hou, Y. Wang, Z. Zhang, X. Wang, and Y. Cao, “SAKMS: Secure Authentication and Key Management Scheme for IETF 6TiSCH,” *IEEE Transactions on Network Science and Engineering*, 2024.
- [3] S. Das and M. Khatua, “Lightweight and Decoupled Distributed Authentication for Multihop IoT Networks,” IIT Guwahati, India.
- [4] “Provably Secure Anonymous Device Authentication Protocol in IoT Environment,” Shanyao Ren, Yizhong LiuD, Beiyuan Yu, Jianwei Liu, *IEEE IoT Journal*, Volume. 11, No. 7, 1 April 2024.
- [5] “Anonymous Authentication in IoT Healthcare Systems.” Mehedi Masud, Gurjot Singh Gaba, Karanjeet Choudhary, M. Shamim Hossain, *IEEE IoT Journals*, Volume. 20, No. 10, Jan 2021
- [6] “LAAKA: Lightweight Anonymous Authentication and Key Agreement.” Hala Ali, Irfan Ahmed, *Volume 140*, May 2024, 103770.
- [7] “Survey for Lightweight Anonymous Authentication.” Jian Zhong, Sheng He, Zhicai Liu and Ling Xiong, 2025.
- [8] “SAKM: PUF-Based UAV Authentication Scheme.” W. Yang, C. Hou, Y. Wang, Z. Zhang, X. Wang and Y. Cao, in *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 3, May-June 2024.